Neural Networks 142 (2021) 573–582

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

An interaction-modeling mechanism for context-dependent Text-to-SQL translation based on heterogeneous graph aggregation

Wei Yu, Tao Chang, Xiaoting Guo, Mengzhu Wang, Xiaodong Wang *

College of Computer, National University of Defense Technology, Kaifu District, Changsha 410073, China

ARTICLE INFO

Article history:
Received 22 January 2021
Received in revised form 25 May 2021
Accepted 9 July 2021
Available online 18 July 2021

Keywords:
Interaction modeling
Context-dependent Text-to-SQL
Heterogeneous graph aggregation

ABSTRACT

For the context-dependent Text-to-SQL task, the generation of SQL query is placed in a multi-turn interaction scenario. Each turn of Text-to-SQL must take historical interactive information and database schema into account. Accordingly, how to encode and integrate these different types of texts (the question sentence, the corresponding SQL query, and database schema) is a tough problem. In previous work, these series of texts are usually concatenated into sequences and encoded by various variants of recurrent neural networks (RNN). However, the RNNs cannot model the intrinsic relationship of the text directly. To this end, we propose an interaction-modeling mechanism to represent and aggregate these texts. Firstly, different types of texts are represented as individual graphs. Then, heterogeneous graph aggregation is used to capture the interactions and aggregate graphs into a holistic representation. Finally, the corresponding SQL query is generated based on the current question and the aggregated information. We evaluate our model on the SparC and CoSQL dataset to demonstrate the benefits of interaction-modeling. Experimentally, our model has a competitive performance and space–time cost.

1. Introduction

Natural language interface to database (NLIDB) has been the holy grail of natural language understanding and human-database interaction for decades (Bais et al., 2019; Hendrix, 1977; Li & Jagadish, 2014; Wang et al., 2020; Woods, 1973). It bridges the gap between humans and databases and provides data support for many application scenarios such as social robot (Ahmad et al., 2017), smart healthcare (Wu et al., 2020), and AI teaching assistants (Kim et al., 2020). The core problem of NLIDB research is how to map natural language utterances to executable SQL queries (Text-to-SQL). At the beginning of the study, researchers focused on accurately translating a sentence into the corresponding SQL query (Yu, Zhang et al., 2018; Zhong et al., 2017). However, people usually ask a series of related questions to achieve a specific goal in real life rather than describing the problem as a complex discourse. Therefore, the context-dependent Text-to-SQL task, represented by SParC (Yu, Zhang, Yasunaga et al., 2019) dataset, is proposed. In this paper, we focus on context-dependent Text-to-SQL translation that makes it possible for a database to understand multiple turns of natural language queries.

Fig. 1 shows a sample in the SparC, where the machine needs to synthesis a SQL query based on the current question, interaction history, and database schema. The multiple turns of interaction mimic the way humans acquire information by asking questions in daily life. There are two main challenges. One is that the text of the current question is not sufficient to generate a complete SQL query. For Q_4 in Fig. 1, the machine has no idea who the *those* in the current question (Q_4) refers to and which table to get the information from without prior knowledge of the previous questions and the database schema. Hence, the task requires the machine to make full use of context to assist in the generation of SQL query for the current turn of question. The other challenge is that it is difficult to encode and integrate the three types of text. As shown in Fig. 1, part *a* is the database schema involved in the interaction which is table-type text. Parts *b* and *c* contain two types of data: question and SQL query. The question is unstructured text and the SQL query is structured text. It is not optimal to simply use a traditional encoder to handle these three distinctive texts. In summary, the setting of context-dependent makes the Text-to-SQL task more difficult and new approaches are urgently needed to address these challenges.

In previous related works, taking the recurrent neural network (RNN) as the encoder is a common practice on the occasion of all information is in text form. Generally, questions and SQL queries are directly encoded by various variants of RNN, and discrete database schema texts are concatenated into a sequence for encoding. Long short term memory (LSTM)

* Corresponding author.
E-mail addresses: yuwei19@nudt.edu.cn (W. Yu), changtao15@nudt.edu.cn (T. Chang), guoxiaoting19@nudt.edu.cn (X. Guo), wangmengzhu19@nudt.edu.cn (M. Wang), xdwang@nudt.edu.cn (X. Wang).

https://doi.org/10.1016/j.neunet.2021.07.014
0893-6080/

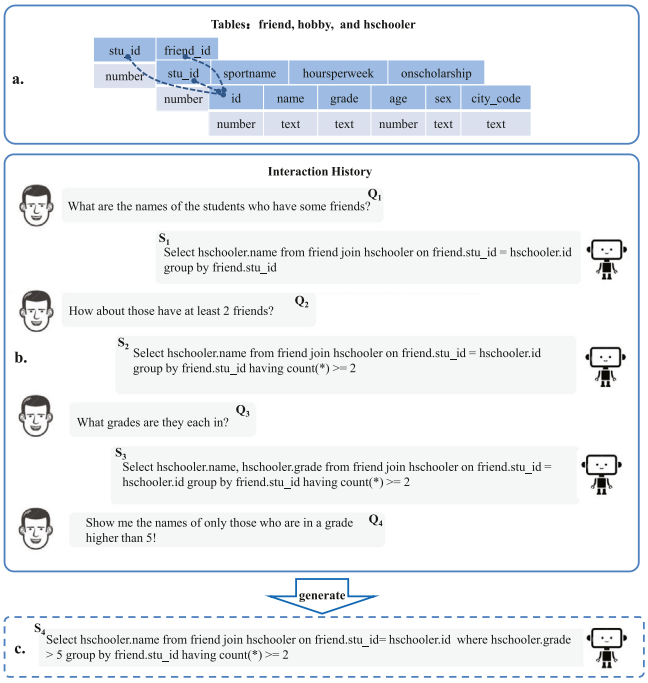

Fig. 1. An example of question sequences from the SParC dataset, where the system needs to generate a corresponding SQL query according to the current question, interaction history, and database schema. Part a is the database schema associated with the question and the dotted lines between tables are foreign key relationships. Part b is a presentation of the current problem (Q_4) and interaction history. Part c is the SQL query generated by the machine combining various information.

(Hochreiter & Schmidhuber, 1997) network and gated recurrent unit network (GRU) (Cho et al., 2014) are used flexibly in Chang et al. (2020), Dong and Lapata (2018), Guo and Gao (2018), Sun et al. (2018), Wang et al. (2018), Xu et al. (2017), Yu, Li et al. (2018), Zhang et al. (2019) and Zhong et al. (2017).

However, these RNN-Encoder based methods have limitations. On the one hand, these methods model all the input text solely in sequence, which may ignore the intrinsic relationship of the text. As shown in Fig. 2, the traditional RNN only encodes the text of the database schema sequentially from left to right without modeling foreign key. On the contrary, foreign-key relationships are extremely important in many cases. Similarly, the significant correlation between nouns, adjectives, and quantifiers in the question and the relevance between the structural blocks in the SQL statement are also very beneficial for the context-dependent Text-to-SQL task. On the other hand, the original encoder–decoder framework of RNN is incapable of systematically employing and aggregating the different kinds of information directly. Concatenating the hidden state of each RNN encoded is an effective approach, but it is too crude. Finding an aggregation method that matches the properties of the data itself is still a worthwhile endeavor.

To alleviate these problems, we propose a novel model called **h**eterogeneous **g**raph **a**ggregation based Text-to-**SQL** generator abbreviated as HGASQL. In particular, HGASQL introduces an interaction-modeling mechanism (see Section 4.2.2 for details) into the traditional encoder–decoder architecture. Firstly, the interaction-modeling mechanism represents three kinds of information as three individual graphs according to the intrinsic relationship of the input text. In order to better capitalize on the intra-actions[1] and inter-actions[2] of the three kinds of graphs,

[1] Influence between nodes within a graph.

[2] Interaction between the two graphs.

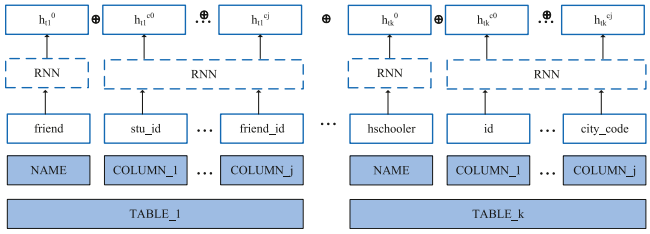

Fig. 2. A schematic of encoding a database schema (Fig. 1.*a*) using the traditional RNN. ⊕ is concatenation operation.

node embedding (Hamilton et al., 2017; Li et al., 2016) and graph aggregation (Zhang et al., 2020) are used to effectively aggregate heterogeneous graphs into a holistic graph representation. Then, the feature of the current question and the aggregated historical information are used comprehensively to decode and generate the corresponding SQL query. The interaction-modeling mechanism essentially lets the decoder more easily keep track of what previous turns have focused on and compensates for the lack of information for the current question.

In summary, our contributions in this paper are three-fold:

- To the best of our knowledge, this is the first work to use graphs to represent all interactive historical information (question, corresponding SQL query, and database schema) in context-dependent Text-to-SQL task.
- We propose HGASQL, a novel model for context-dependent Text-to-SQL generation by employing the flexible interaction-modeling mechanism to capture the interactions across heterogeneous graphs and aggregate them into a holistic representation.
- We conduct relatively extensive experiments on the SparC and CoSQL dataset. Experimental results show that HGASQL has good convergence, and achieves a certain improvement over the baseline.

2. Related work

With the rapid development of technology, various models have been proposed over the years to translate the natural language to SQL query. Depending on the constrained setting of the task, these studies can be classified into three categories: single-table, cross-domain, and context-dependent.

Single-table Text-to-SQL constrained the problem by two factors: each question is only addressed by a single table, and the table is known. Even though the scope is constrained, the task is still very challenging because the tables and questions are very diverse. In the past, many approaches were proposed for this task. They primarily share the RNN-based encoder–decoder architecture. The main difference lies in the detailing of the decoding stage. Some early work tried to decode sequentially (Dong & Lapata, 2016; Zhong et al., 2017). However, it is found challenging to guarantee the output correct in syntax. Later more work breaks down a SQL query into several parts like SELECT column, WHERE column, WHERE operator, WHERE value, etc. and have sub-decoders for them (Guo & Gao, 2018; He et al., 2019; Hwang et al., 2019; Lyu et al., 2020; Sun et al., 2018; Xu et al., 2017).

Cross-domain means that every query is conditioned on a multi-table database schema, and the databases do not overlap between the train and test sets. This setting makes the SQL queries more complex and the database schema more interleaved. For generating complex SQL queries, some works use the syntax tree or semSQL to guide the generation. Alvarez-Melis and Jaakkola (2017) proposed a doubly recurrent neural network

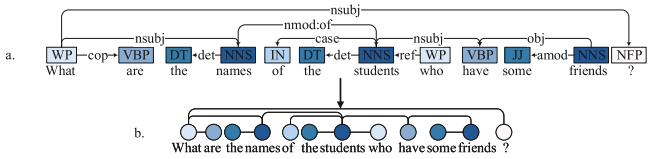

Fig. 3. An example of constructing a question graph through dependency parsing graph.

model to combine inside each cell (node) to generate an output. SyntaxSQLNet (Yu, Yasunaga et al., 2018) employs a SQL specific syntax tree-based decoder with SQL generation path history and table-aware column attention encoders. In order to deal with the complex database schema, GNN is used for this task. In the work (Huo et al., 2019), the structure of the database schema is encoded with a GNN to guide the decoding. To make a more contextually-informed selection of database constants, the Global GNN (Bogin et al., 2019) model uses GNN to encode database schema for global reasoning.

Context-dependent setting requires the model not only to focus on the precise generation of SQL queries, but also to consider the comprehensive utilization of contextual information. The context-dependent Text-to-SQL task is an advanced version of the single-table and cross-domain Text-to-SQL. The first publicly available paper about context-dependent Text-to-SQL task is EditSQL (Zhang et al., 2019). The framework of EditSQL consists of an utterance-table encoder to encode the user question and database schema at each turn and takes turn attention to incorporate the recent interaction history for decoding. It uses RNN to encode all the textual information and employs the attention mechanism to model the interactions between texts. Another related work is IGSQL (Cai & Wan, 2020). IGSQL proposes a database schema interaction graph encoder to utilize historical information of database schema items and a gate mechanism to weigh the importance of different vocabularies in decoding phase. It takes RNN to encode question and SQL query and employs an interaction graph to model context consistency of an interaction. Differently, our approach uses three types of graphs to represent and construct the internal relationships of the input texts and utilizes graph aggregation technology to integrate information into a holistic graph representation.

Our work is also related to recently proposed approaches to graph aggregation. The paper (Hamilton et al., 2017) focuses on the relationship between the nodes in each graph and proposes three aggregation functions: Mean aggregator, LSTM aggregator, and Pooling aggregator. The global–local aggregation network is proposed to aggregate the information between graphs in the work (Zhang et al., 2020). In our work, we use the Max-Pooling aggregator to calculate the initial value of each node and follow the global–local aggregation network to integrate the interactive information.

3. Graph representations

3.1. Question graph

In the Text-to-SQL task, the nouns in the question are usually related to the column names, and the adjectives and quantifiers are related to specific aggregate functions or column values. Capturing these task-related words and their dependencies is helpful to better encode the question text. To this end, as shown in Fig. 3.*a*, Stanford CoreNLP toolkit (Manning et al., 2014) is used to perform dependency parsing on the text of the question. And then a question graph G_Q is constructed according to the dependency parsing graph.

3.2. SQL graph

A basic SQL query consists of three parts: SELECT clause, FROM clause, and WHERE clause, it can be easily converted into a graph. For specific query purposes, various functions are used to restrict these three clauses and multiple SQL query statements are often nested. It is difficult to transform these complex SQL queries into graphs sequentially. To solve this problem, we propose a non-nested graph generation algorithm 1. The first step is to traverse the SQL query $S = \{S_1, S_2, \ldots, S_n\}$ and find the position of the SELECT keyword. Then S is divided into several sections by the index of the SELECT. Each section is a basic SQL statement ending with a connection keyword. After that, the basic SQL statements and connection keywords are put into queue *Que*, respectively. Finally, traversing the *Que* to construct the SQL graph. If the basic SQL query is taken out, the keywords and values in the query are treated as nodes and connecting these nodes to construct a sub-graph G' according to their inherent relationship. Then, connect the generated sub-graph with the previous sub-graph through the connection node. If a connection keyword is taken out, generate a connection node, and connect it to the previous sub-graph. Notably, if the last node of the sub-graph before the connection keyword is an independent non-keyword node, the node needs to be connected to the SELECT node.

3.3. Database schema graph

Conceptually, a schema is a set of interrelated database objects, such as tables, table columns, data types of the columns, indexes, foreign keys, and so on. These objects can be connected to form a database schema graph naturally, because the columns make up the tables and the foreign keys refer to tables and columns. As shown in Fig. 4, we firstly generate nodes for the header and column of each table and connect the nodes of all columns with their header node. After that, the foreign key nodes of each table are connected to form a database schema graph G_{DB}.

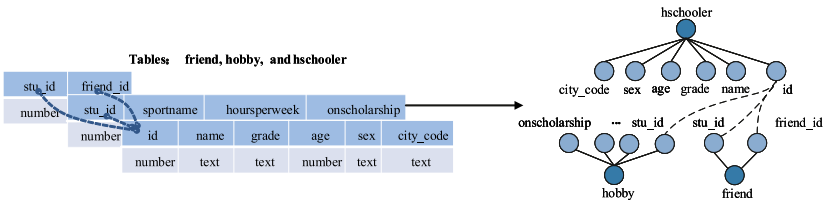

Fig. 4. Schematic diagram of constructing a database schema graph.

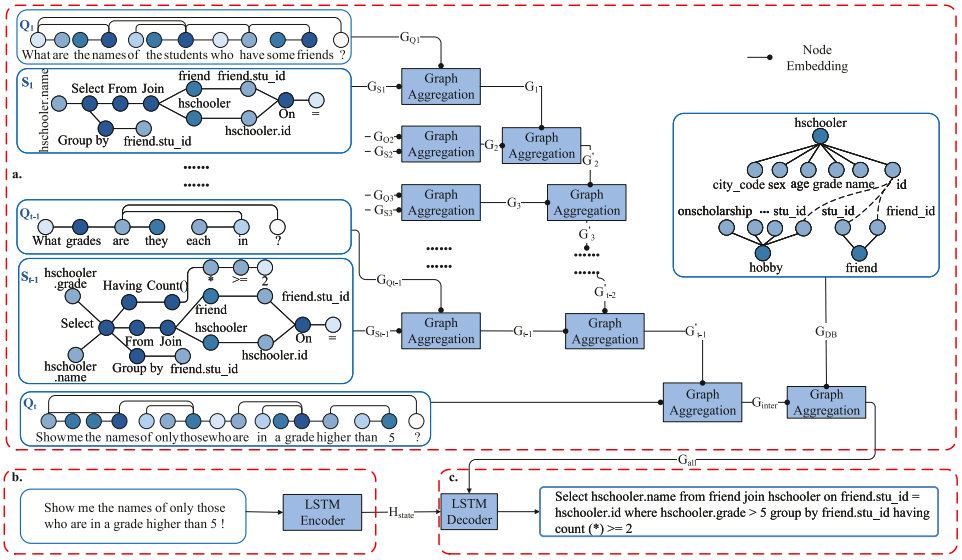

Fig. 5. Schematic of our proposed HGASQL. Part *a* is the illustration of interaction-modeling, part *b* is a LSTM encoder, and part *c* is a LSTM decoder.

Algorithm 1: Non-nested SQL Graph Generation

Input: SQL $S = \{S_1, S_2, \cdots, S_n\}$, Keywords Set $K = \{SELECT, WHERE, \cdots, IN\}$, Queue *Que*
Output: SQL Graph G_S

```
for i=1:n do
    if S_i == SELECT and i != 1 then
        Put S[1 : i − 2] in Que;
        Put S[i − 1] in Que;
        i → index;
    else
        continue;
    end
end
Put S[index :] in Que;
for j=1:len(Que) do
    if j% 2==1 then
        Construct the subgraph G′;
        if j==1 then
            G′ → G_S
        else
            Connect the SELECT node of G′ to the Que[j − 1]
            node of G_S;
        end
    else
        if Que[j − 1][−2] not in K then
            Connect the Que[j] node to the SELECT node of
            the previous subgraph ;
        else
            Connect the Que[j] node to the Que[j − 1][−1]
            node of the previous subgraph;
        end
    end
end
```

4. Methodology

4.1. Task definition

The context-dependent Text-to-SQL task aims to combine the current question with previous turns of interaction information and database schema to generate corresponding SQL query. Let $I = [(Q_1, S_1), \ldots, (Q_{t-1}, S_{t-1}), (Q_t, S_t)]$ denote the interaction history in a context-dependent setting, where Q_t and S_t are the tth question and corresponding SQL query, respectively. Let $D = [t_1, t_2, \ldots, t_n]$ denote the tables of current database, where n is the number of table. At each turn i, the task can be formulated as a conditional text generation problem. The goal is to generate S_i given the three kinds of information: the current utterance Q_i, the interaction history $I_{i-1} = [(Q_1, S_1), (Q_2, S_2), \ldots, (Q_{i-1}, S_{i-1})]$ and database schema D.

4.2. Architecture

As shown in Fig. 5, HGASQL builds upon a standard attention-based encoder–decoder architecture comprised of (a) an *interaction modeling* aggregating the interaction history into a holistic representation for decoding, (b) an *encoder* that operates token-by-token over the current question, and (c) a *decoder* taking into account the question, the database schema, and the previously generated query to generate.

4.2.1. Encoder

Let $Q_t = [q_1, q_i, \ldots, q_n]$ denote an question sequence for tth turn where q_i is the ith token. An LSTM network is employed to encode the question and the hidden states $H^Q = [h_1^Q, h_2^Q, \ldots, h_n^Q]$ of all encoding steps are kept. For brevity, we do not write out the LSTM equations, which are described by Hochreiter and Schmidhuber (1997).

4.2.2. Interaction-modeling mechanism

The main challenges of cross-domain context-dependent Text-to-SQL come from the multi-source nature of the input information and the difficulty in capturing information that is conducive to the current turn of SQL generation in multiple turns of interaction. To this end, we propose an interaction-modeling mechanism, it can integrate multiple types of historical information to make up for the lack of input information. The interaction-modeling includes two aspects: one is to model the intra-actions of nodes in each graph by node embedding, and the other is to model the inter-actions of different kinds of information in each turn using heterogeneous graph aggregation.

Node embedding. As a common practice, we leverage graph neural networks (GNN) to aggregate information in local neighborhood nodes and exploit the rich information inherent in graph structure for each graph. According to the constructed graph $G = (V, E)$, we first use an LSTM network to encode the text attributes of each node $\forall v \in V$ and obtain the feature vector a_v to initialize the node. Then, we re-compute the feature vector using K-hop neighbor's information (Hamilton et al., 2017). In this process, we repeatedly use the following method K times:

$$\begin{aligned} h_v^0 &= a_v, \forall v \in V \\ h_{N(v)}^k &= MAX\left(\left\{\delta\left(W_{pool} h_u^{k-1} + b\right), \forall u \in N(v)\right\}\right) \\ h_v^k &= \sigma\left(W^k [h_v^{k-1}; h_{N(v)}^k]\right) \end{aligned} \tag{1}$$

where $\forall k \in \{1, \ldots, K\}$, $N(v)$ is the neighbor nodes set of v, MAX denotes the element-wise max-pooling operator, W_{pool}, W^k are weight matrices, b is bias term and δ, σ are nonlinear activation functions. h_v^k is the representation of node v which aggregates the information of the neighborhood node with depth K.

Heterogeneous graph aggregation. To capture the inter-actions across heterogeneous graphs, we follow the work (Zhang et al., 2020) to transform heterogeneous graphs into an aggregated graph. Given the query graph $G_q = \{v_1^q, v_2^q, \ldots, v_n^q\}$, and the context graph $G_e = \{v_1^e, v_2^e, \ldots, v_m^e\}$, we aim to obtain an aggregated graph G_{agg} with the same structure as G_q. The process of graph aggregation can be formally defined as: $GA(G_e, G_q) \rightarrow G_{agg}$. Let h_{v^q} and h_{v^e} denote the representation for the query node v^q and the context node v^e.

Firstly, global-average pooling is performed on the query graph G_q to get the global representation $h_{v^{q*}}$.

$$h_{v^{q*}} = \frac{1}{n} \sum_{i=1}^{n} h_{v_i^q} \tag{2}$$

Then, a gate function is used to capture globally relevant information in the context graph G_e and get the query-aware context feature.

$$g_j = \theta\left(W_g [h_{v^{q*}}; h_{v_j^e}] + b_g\right) \tag{3}$$

$$h_{v_j^{eq}} = (1 - g_j)(W_q h_{v^{q*}} + b_q) + g_j (W_e h_{v_j^e} + b_e) \tag{4}$$

where W_g, W_q and W_e are weight matrices, θ is a sigmoid function, b_g, b_q and b_e are bias terms. g_j is the relevance between the global query graph representation $h_{v^{q*}}$ and one context graph node $h_{v_j^e}$. Eq. (4) projects the original representations into a query-context joint space.

Finally, node-level additive attention (Bahdanau et al., 2015) is used to filter important features related to each individual query node. Specifically, we compute the final aggregated node vector $h_{v_i^{agg}}$ as the following:

$$o_{i,j} = tanh(W_o [h_{v_i^q}; h_{v_j^{eq}}] + b_o) \tag{5}$$

$$\bar{o}_{i,j} = \frac{exp(W_a o_{i,j})}{\sum_k exp(W_a o_{i,k})} \tag{6}$$

$$h_{v_i^{agg}} = h_{v_i^q} + \sum_{j=1}^{m} \bar{o}_{i,j} h_{v_j^{eq}} \tag{7}$$

The node-level relevance $\bar{o}_{i,j}$ between one query node $h_{v_i^q}$ and one updated context node $h_{v_j^{eq}}$ is computed by Eqs. (5) and (6). In Eq. (7), all locally relevant information have been distilled and summed up to obtain $h_{v_i^{agg}}$.

Aggregating all graphs. As shown in Fig. 2, the node embedding represented by the solid circle is performed before each graph aggregation. For graph aggregation, turn-level graph aggregation is firstly performed. The question graph G_{Q_i} and SQL graph G_{S_i} of each turn in the interaction history are aggregated into SQL-question Graph, $GA(G_{Q_i}, G_{S_i}) \rightarrow G_i$. The SQL graph is used as the query graph to distill relevant information from the question graph. Then, graph aggregation between each turn is performed. Each turn of G_i is taken as the query graph to aggregate the previous turn of G_{i-1} to obtain the multi-turn graph, $GA(G_{i-1}, G_i) \rightarrow G'_i$. After that, the tth question graph G_{Q_t} and multi-turn graph G'_{t-1} are aggregated into interaction-graph, $GA(G_{Q_t}, G'_{t-1}) \rightarrow G_{inter}$. Finally, database schema graph G_{DB} and interaction-graph G_{inter} are aggregated into G_{all}, $GA(G_{DB}, G_{inter}) \rightarrow G_{all}$.

4.2.3. Decoder

Another LSTM network is taken as a decoder. The state h_0^D of the decoder is initialized by the final state of the above encoder, $h_0^D = h_n^Q$. In the decoding stage, we comprehensively consider the aggregated graph information and the hidden state of the current question. Formally, at step m, the node-level attention f_{na} (i.e. Eqs. (5), (6), and (7)) is applied to distill relevant information and obtain the context feature C_m^D:

$$C_m^D = [f_{na}(h^{G_{all}}, h_m^D); f_{na}(H^Q, h_m^D)] \tag{8}$$

where $h^{G_{all}}$ denotes all nodes features of G_{all}. Then, the context feature C_m^D is put into the decoder.

$$h_{m+1}^D = LSTM(h_m^D, C_m^D) \tag{9}$$

In the output layer, our decoder chooses to generate a SQL keyword or a column header. We use separate layers to score SQL keywords and column headers, and finally use the softmax operation to generate the output probability distribution:

$$\begin{aligned} \eta_m &= tanh([h_{m+1}^D; C_m^D] W_m) \\ \chi^{SQL} &= \eta_m W_{SQL} + b_{SQL} \\ \chi^{column} &= \eta_m W_{column} h^{G_{DB}} \\ P(y_m) &= softmax([\chi^{SQL}; \chi^{column}]) \end{aligned} \tag{10}$$

where W_m, W_{SQL}, W_{column} are linear transformation matrices, b_{SQL} is a bias term, $h^{G_{DB}}$ is all nodes features of database schema graph G_{DB}.

5. Experiments

5.1. Datasets and evaluation metrics

We train and evaluate our model on the SParC (Yu, Zhang, Yasunaga et al., 2019) and CoSQL (Yu, Zhang, Er et al., 2019). The SParC dataset is a widely used dataset dedicated to evaluating context-dependent Text-to-SQL task. It consists of 4298 coherent question sequences and 12k+ individual questions annotated with SQL queries, obtained from controlled user interactions with 200 complex databases over 138 domains. CoSQL is a general-purpose database querying dialog dataset, which contains the following tasks: context-dependent Text-to-SQL, natural language response generation, and user dialog act prediction.

We follow the evaluation metrics in Yu, Zhang, Yasunaga et al. (2019) to measure the query synthesis accuracy: question match and interaction match. Question match is the exact set matching score over all questions and interaction match is the exact set matching score over all interactions. The exact set matching score is 1 for each question only if all predicted SQL clauses are correct, and 1 for each interaction only if there is an exact set match for every question in the interaction.

Table 1
Performance of various methods on SparC and CoSQL dev set.

Model	SparC		CoSQL	
	Question match	Interaction match	Question match	Interaction match
CD-Seq2Seq (Yu, Zhang, Yasunaga et al., 2019)	21.9	8.1	13.8	2.1
SyntaxSQL-con (Yu, Zhang, Yasunaga et al., 2019)	18.5	4.3	15.1	2.7
EditSQL (Zhang et al., 2019)	31.4	14.7	21.2	6.3
EditSQL+Editing	40.6	17.3	29.5	8.6
EditSQL+BERT	40.4	18.1	30.7	8.8
EditSQL+BERT+Editing	47.2	29.5	39.9	12.3
IGSQL+BERT (Cai & Wan, 2020)	50.7	**32.5**	44.1	15.8
HGASQL	35.9	16.8	25.6	10.3
HGASQL+Editing	41.8	18.7	33.5	9.2
HGASQL+BERT	41.7	18.5	33.1	9.8
HGASQL+BERT+Editing	**51.5**	31.6	**44.9**	**16.2**

5.2. Baselines

5.2.1. CD-Seq2Seq

CD-Seq2Seq (Yu, Zhang, Yasunaga et al., 2019) is a cross-domain Seq2Seq based Text-to-SQL model extended with the turn-level history encoder. In particular, the input of the turn-level history encoder is the concatenation of the hidden states of the historical interaction encoded by a bi-LSTM and the word embedding of the current turn question.

5.2.2. SyntaxSQL-con

SyntaxSQLNet (Yu, Yasunaga et al., 2018) is a syntax tree based neural model for the complex and cross-domain context-independent text-to-SQL task. The SyntaxSQL-con (Yu, Zhang, Yasunaga et al., 2019) extends SyntaxSQLNet by providing the decoder with encoding of the previous question as additional contextual information.

5.2.3. EditSQL

EidtSQL (Zhang et al., 2019) takes a query editing mechanism to reuse generation results at the token level and uses an utterance-table encoder and a table-aware decoder to incorporate the context of the user utterance and the table schema.

5.2.4. IGSQL

IGSQL (Cai & Wan, 2020) is also an encoder–decoder framework with attention mechanism that adopts a database schema interaction graph encoder to model context consistency of an interaction, a text encoder to captures historical information of user inputs, a co-attention module to updates outputs of text encoder and database schema interaction graph encoder, and a decoder with a gated mechanism to weight the importance of different vocabularies.

5.3. Implementation details

Our model is implemented in PyTorch 1.2.0. The system is Ubuntu 18.04 with two 2080ti graphic cards and 32G of flash memory. All LSTM layers have 512 hidden size, and we use 1 layer for encoder LSTM, and 2 layers for decoder LSTM. The initial dimensions of the question graph and SQL graph are 64, and the dimension of the database schema graph is 256. The depth of node embedding K is 4. We use pretrained 300-dimensional GloVe (Pennington et al., 2014). The batch size is set to 16 and we use Adam optimizer (Kingma & Ba, 2015) with the setting $\beta_1 = 0.9, \beta_2 = 0.999$. Learning rate is 3×10^{-4}. For the BERT option of the comparative experiment, we use the pretrained small uncased BERT model with 768 hidden size.[3] The editing mechanism is based on work (Zhang et al., 2019).

[3] https://github.com/google-research/bert.

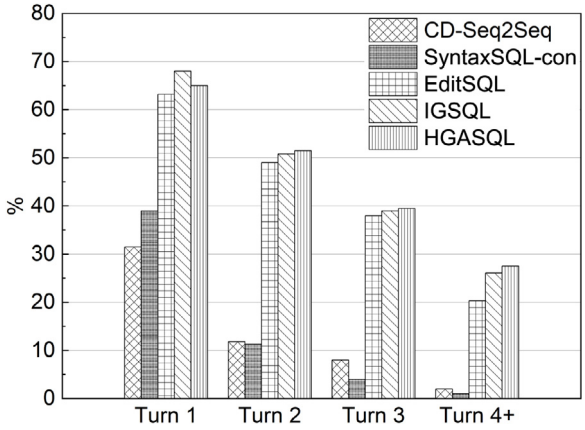

Fig. 6. Performance split by different turns on SparC dev set.

5.4. Results and analysis

5.4.1. Overall results

As can be seen from Table 1, our model without any assistance outperforms other baselines, achieving 35.9% question matching accuracy and 16.8% interaction matching accuracy. In addition, compared with the baselines(CD-Seq2Seq and SyntaxSQL-con) proposed in the SparC dataset, the performance of our model is almost doubled. Moreover, our model is 4% and 2% better than EditSQL in question matching and interaction matching, respectively. In addition, our model outperforms IGSQL on the three indicators of the SparC and CoSQL. In summary, our model based on heterogeneous graph aggregation has advantages over traditional RNN-based models and achieves a competitive performance on two widely used datasets.

5.4.2. Performance analysis

To better understand how our model improves performance, we make various aspects of performance analysis: performance of different turns, attention weights of interaction-modeling mechanism, convergence speed and memory consumption.

Performance of different turns. In order to better analyze the performance, we have made statistics on the accuracy of the models in different turns. Fig. 6 shows the performance split by turns. What stands out in Fig. 6 is that the questions asked in later turns are more difficult to answer given longer context history. A possible explanation is that pronouns and clauses that refer to previous descriptions in later turns of questions increase the difficulty of the task. It can also be seen from Fig. 6, our model has a good performance in each turn, especially in the later turns. This demonstrates our model is more competitive in answering hard and extra hard questions.

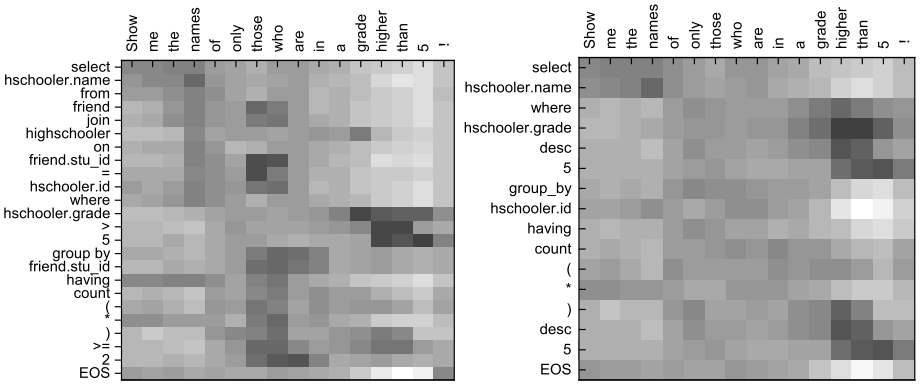

Fig. 7. Attention over the source sequence visualized with(*a*) and without(*b*) the interaction-modeling mechanism. Darker cells mean higher values.

Table 2
Ablation study on SparC and CoSQL dataset. Numbers in brackets are performance differences compared to HGASQL.

Model	SparC		CoSQL	
	Question match	Interaction match	Question match	Interaction match
HGASQL	51.5	31.6	44.9	16.2
w/o Question graph	49.2(−2.3)	29.1(−2.5)	42.4(−2.5)	13.1(−3.1)
w/o SQL graph	45.4(−6.1)	26.3(−5.3)	38.4(−6.5)	11.8(−4.4)
w/o Database schema graph	44.3(−7.2)	25.4(−6.2)	37.9(−7.0)	11.6(−4.6)

Attention weights of interaction-modeling mechanism. To better understand how our model improves performance, we use the attention heatmap to visualize the attention weights of interaction-modeling mechanism. As shown in Fig. 7, it is the attention weights of the 4-th turn of question in Fig. 1. In the left sub-figure *a*, attention weights are shown when the interaction-modeling mechanism is utilized, while in the right sub-figure *b* standard attention is used. As can be seen from the figure *b*, the standard attention only focuses on the text of the current question (e.g. the information of *grade higher than 5* is used twice) and the word (*those, who*) and the generated SQL query do not have a high attention score. Differently, in the figure *b*, a significant high attention score is presented between the word (*those, who*) and the subset (e.g. *having count(*) >= 2*') of generated SQL query. Those subset SQL queries are segments of the previous turn's SQL query. This demonstrates interaction-modeling mechanism can drive the attention weights to be more focused on the relevant source token(s) and effectively take historical information into account to generate SQL queries.

Convergence speed. Convergence is an important performance indicator of the neural network model. To compare the convergence of our model and the baseline, the loss, question match, and time per step are presented in Fig. 8. Sub-figure *a* shows that each step of our HGASQL model takes about 0.5–1 min which is much lower than the SOTA (EditSQL and IGSQL). And from sub-figures *a* and *c*, it can be seen that whether it is step-loss or epoch-loss, the loss value of the HGASQL model is almost at the bottom of the chart and is lower than other models. Moreover, the sub-figures *c* and *d* illustrate that our HGASQL model has better accuracy and completes the training after 19 epochs. In summary, HGASQL has certain competitiveness in terms of model convergence while ensuring the accuracy of the model.

Memory consumption. As shown in Fig. 9, graphic Ram and flash memory are used to analyze the physical cost of the model. It can be seen that the graphic Ram consumption of HGASQL is slightly higher than EditSQL and lower than IGSQL at about 3.5G. Although the flash memory consumption of HGASQL and EditSQL fluctuates greatly, they are about the same magnitude as the stable IGSQL.

Through the performance analysis of the above four aspects, the following conclusions can be drawn: (a) HGASQL has a competitive performance in each turn of SQL translation, (b) HGASQL can make better use of context information to make up for the lack of information in the current turn, (c) While ensuring the performance of the model, HGASQL consumes time and space within an acceptable range.

5.4.3. Ablation study

In order to verify the usefulness of our three types of graphs, we conduct several ablation experiments as follows: w/o Question Graph, w/o SQL Graph and w/o Database Schema Graph. It should be noted that the w/o in the experiment indicates that the corresponding graph structure is replaced by RNN, rather than completely removed.

Table 2 shows the results of ablation experiments. It can be found that under the three experimental settings, the performance of the model decreases to varying degrees. Especially in the case of without SQL Graph and without Database Schema Graph, the degradation of model performance is particularly obvious. These results indicate that three types of graph are very helpful.

5.4.4. Further discussion

As aforementioned, we represent three kinds of information as individual graphs. Therefore, there are many different combinations for the aggregation of these graphs. We design three sets of comparative experiments to explore which kind of aggregation is most beneficial to the task. One is which graph is used as the query graph during aggregation, another is whether the database graph needs to be aggregated with the previous turns of information, and the last is whether aggregation is needed between each turn.

The selection of the query graph. To explore the best options for the query graph, we set up three experiments, i.e., SQL graph as query graph, question graph as query graph, and database schema graph as query graph. Specifically, in the experiment using the SQL graph as a query graph, we always treat the SQL graph in any graph aggregation operation as a query graph and distill

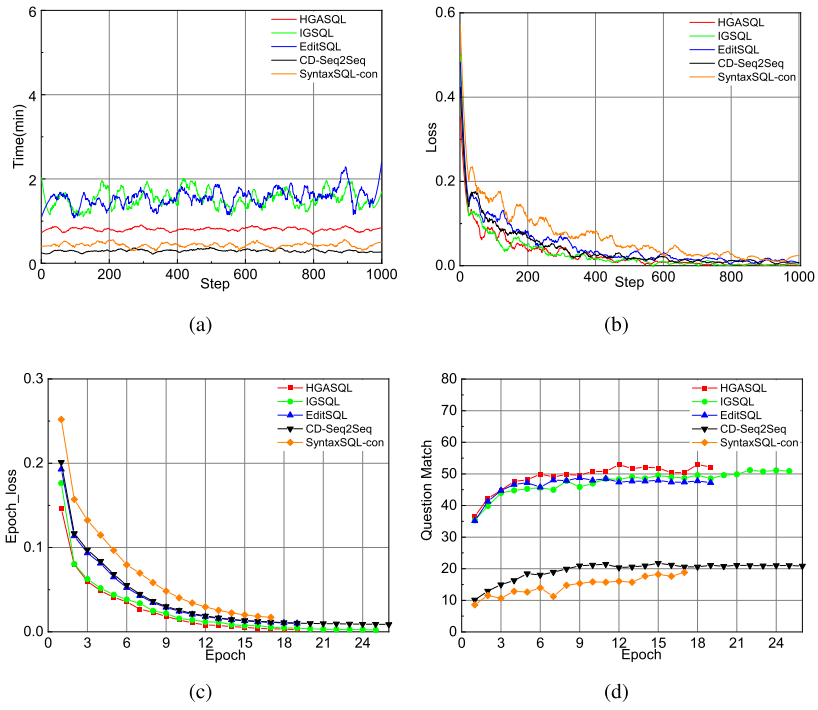

Fig. 8. The epoch-loss, epoch-accuracy, step-loss, and step-time of our model and baselines on SParC dev set.

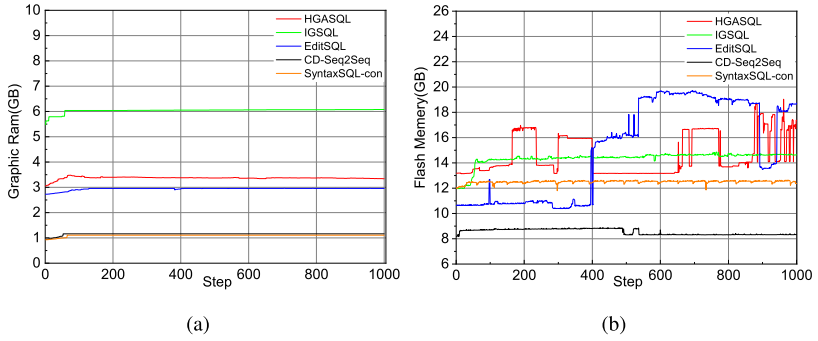

Fig. 9. The graphic Ram and flash memory consumption of our model and baselines on SParC dev set.

the relevant information from other graphs. The remaining two experiments are also set up in this way. Fig. 10.*a* presents the results from the above three experiments. We observe that the highest accuracy is obtained by the group treating the SQL graph as a query graph.

The necessity of database schema graph. To explore whether or not a database schema graph should be required, and if so, in what phase. Firstly, we set up an experiment whose graph operations always need to be aggregated with the database schema graph. Secondly, we perform the aggregation with the database schema graph only once after the other information is aggregated as described in Section 4.2.2. The last experiment does not use any database schema information. The results are shown in Fig. 10.*b*. From the chart, we can draw the following two conclusions: One is that the full participation of the database schema graph cannot significantly improve the experimental result, because it obtains an accuracy of 30.7% which is 5.2% less than the experiment using the database schema graph only once. The other is that the database schema graph is good for the task since the experiment without using the database schema graph has the worst accuracy, only 12.5% of the question matching and 5.2% of the interaction matching are achieved.

Table 3
Experimental results with or without turn level aggregation.

	Question match	Interaction match
Turn level aggregation	35.9	16.8
None	25.7	11.5

Turn level aggregation or not. As shown in Table 3, we conduct experiments with or without turn level aggregation. It is worth mentioning that the experiment without turn level aggregation means to integrate the aggregated graph of each turn into the context feature C_m^D following Eq. (8). In Table 3, there is a clear trend of decreasing as the turn level aggregation is removed.

The above three sets of experimental results show that choose a reasonable aggregation method can improve model performance and our method is the best.

6. Conclusion

In this paper, we propose a comprehensive information integration framework, named HGASQL, to generate complex SQL queries for context-dependent Text-to-SQL task. As far as we

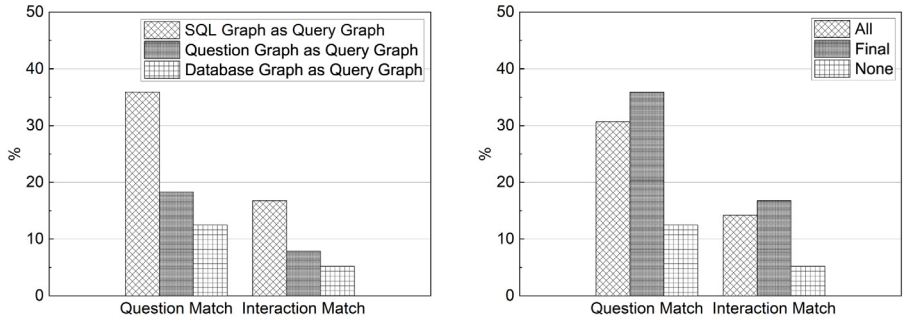

Fig. 10. Experimental results of different query graphs and different usages of database schema graph.

know, this is the first piece of work that explores graph aggregation for the this task. Extensive studies are conducted on the SParC and CoSQL dataset to evaluate the proposed model, and the results confirm its effectiveness with very favorable performance over several state-of-the-art methods. Although some progress has been made, it is still far from free communication between man and database. In the future work, we will continue to follow up the work related to GNN and apply these techniques to the task of Text-to-SQL translation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We thank the anonymous reviewers for their thoughtful detailed comments.

References

Ahmad, M. I., Mubin, O., & Orlando, J. (2017). Adaptive social robot for sustaining social engagement during long-term children–robot interaction. *International Journal of Human–Computer Interaction*, *33*(12), 943–962. http://dx.doi.org/10.1080/10447318.2017.1300750.

Alvarez-Melis, D., & Jaakkola, T. S. (2017). Tree-structured decoding with doubly-recurrent neural networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net.

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. In *3rd International Conference on Learning Representations, ICLR 2015* San Diego, CA, USA: May 7-9, 2015 Conference Track Proceedings.

Bais, H., Machkour, M., & Koutti, L. (2019). An independent-domain natural language interface for multimodel databases. *The International Journal of Computational Intelligence Studies*, *8*(3), 206–222. http://dx.doi.org/10.1504/IJCISTUDIES.2019.102547.

Bogin, B., Gardner, M., & Berant, J. (2019). Global Reasoning over Database Structures for Text-to-SQL Parsing. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019* (pp. 3657–3662).

Cai, Y., & Wan, X. (2020). IGSQL: database schema interaction graph based neural model for context-dependent text-to-SQL generation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020* (pp. 6903–6912). Association for Computational Linguistics.

Chang, S., Liu, P., Tang, Y., Huang, J., He, X., & Zhou, B. (2020). Zero-shot text-to-SQL learning with auxiliary task. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, the Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, the Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020* (pp. 7488–7495). AAAI Press.

Cho, K., van Merrienboer, B., Gülçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, a Meeting of SIGDAT, a Special Interest Group of the ACL* (pp. 1724–1734). ACL.

Dong, L., & Lapata, M. (2016). Language to logical form with neural attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics.

Dong, L., & Lapata, M. (2018). Coarse-to-fine decoding for neural semantic parsing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 1: Long Papers* (pp. 731–742). Association for Computational Linguistics.

Guo, T., & Gao, H. (2018). Bidirectional attention for SQL generation. CoRR, abs/1801.00076, URL: http://arxiv.org/abs/1801.00076.

Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017* Long Beach, CA, USA: (pp. 1024–1034).

He, P., Mao, Y., Chakrabarti, K., & Chen, W. (2019). X-SQL: reinforce schema representation with context. CoRR, abs/1908.08113, URL: http://arxiv.org/abs/1908.08113.

Hendrix, G. G. (1977). Human engineering for applied natural language processing. In *Proceedings of the 5th International Joint Conference on Artificial Intelligence. Cambridge, MA, USA, August 22-25, 1977* (pp. 183–191). William Kaufmann.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Huo, S., Ma, T., Chen, J., Chang, M., Wu, L., & Witbrock, M. (2019). Graph enhanced cross-domain text-to-SQL generation. In *Proceedings of the Thirteenth Workshop on Graph-Based Methods for Natural Language Processing, TextGraphs@EMNLP 2019, Hong Kong, November 4, 2019* (pp. 159–163). Association for Computational Linguistics.

Hwang, W., Yim, J., Park, S., & Seo, M. (2019). A comprehensive exploration on wikisql with table-aware word contextualization. CoRR, abs/1902.01069, URL: http://arxiv.org/abs/1902.01069.

Kim, J., Merrill, K., Xu, K., & Sellnow, D. D. (2020). My teacher is a machine: Understanding students' perceptions of AI teaching assistants in online education. *International Journal of Human–Computer Interaction*, *36*(20), 1902–1911. http://dx.doi.org/10.1080/10447318.2020.1801227.

Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015* San Diego, CA, USA: May 7-9, 2015, Conference Track Proceedings, URL: http://arxiv.org/abs/1412.6980.

Li, F., & Jagadish, H. V. (2014). Constructing an interactive natural language interface for relational databases. *Proc. VLDB Endow.*, *8*(1), 73–84. http://dx.doi.org/10.14778/2735461.2735468.

Li, Y., Tarlow, D., Brockschmidt, M., & Zemel, R. S. (2016). Gated graph sequence neural networks. In Y. Bengio, & Y. LeCun (Eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. URL: http://arxiv.org/abs/1511.05493.

Lyu, Q., Chakrabarti, K., Hathi, S., Kundu, S., Zhang, J., & Chen, Z. (2020). *Hybrid Ranking Network for Text-to-SQL: Technical Report MSR-TR-2020-7*, Microsoft Dynamics 365 AI, URL: https://www.microsoft.com/en-us/research/publication/hybrid-for-text-to-sql/.

Manning, C. D., Surdeanu, M., Bauer, J., Finkel, J., Bethard, S. J., & McClosky, D. (2014). The stanford corenlp natural language processing toolkit. In *Association for Computational Linguistics (ACL) System Demonstrations* (pp. 55–60). URL: http://www.aclweb.org/anthology/P/P14/P14-5010.

Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014* Doha, Qatar, a Meeting of SIGDAT, a Special Interest Group of the ACL, (pp. 1532–1543).

Sun, Y., Tang, D., Duan, N., Ji, J., Cao, G., Feng, X., Qin, B., Liu, T., & Zhou, M. (2018). Semantic parsing with syntax- and table-aware SQL generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 1: Long Papers* (pp. 361–372). Association for Computational Linguistics.

Wang, W., Tian, Y., Wang, H., & Ku, W. (2020). A natural language interface for database: Achieving transfer-learnability using adversarial method for question understanding. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020* (pp. 97–108). IEEE, http://dx.doi.org/10.1109/ICDE48307.2020.00016.

Wang, W., Tian, Y., Xiong, H., Wang, H., & Ku, W. (2018). A transfer-learnable natural language interface for databases. CoRR, abs/1809.02649, URL: http://arxiv.org/abs/1809.02649.

Woods, W. A. (1973). Progress in natural language understanding: an application to lunar geology. In *AFIPS Conference Proceedings: vol. 42, American Federation of Information Processing Societies: 1973 National Computer Conference, 4-8 June 1973, New York, NY, USA* (pp. 441–450). AFIPS Press/ACM.

Wu, K., Liu, C., & Calvo, R. A. (2020). Automatic nonverbal mimicry detection and analysis in medical video consultations. *International Journal of Human–Computer Interaction*, *36*(14), 1379–1392. http://dx.doi.org/10.1080/10447318.2020.1752474.

Xu, X., Liu, C., & Song, D. (2017). Sqlnet: Generating structured queries from natural language without reinforcement learning. CoRR, abs/1711.04436, URL: http://arxiv.org/abs/1711.04436.

Yu, T., Li, Z., Zhang, Z., Zhang, R., & Radev, D. R. (2018). TypeSQL: Knowledge-based type-aware neural text-to-SQL generation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 2 (Short Papers)* (pp. 588–594). Association for Computational Linguistics.

Yu, T., Yasunaga, M., Yang, K., Zhang, R., Wang, D., Li, Z., & Radev, D. R. (2018). Syntaxsqlnet: Syntax tree networks for complex and cross-domain text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018* (pp. 1653–1663). Association for Computational Linguistics.

Yu, T., Zhang, R., Er, H., Li, S., Xue, E., Pang, B., Lin, X. V., Tan, Y. C., Shi, T., Li, Z., Jiang, Y., Yasunaga, M., Shim, S., Chen, T., Fabbri, A., Li, Z., Chen, L., Zhang, Y., Dixit, S., ... Radev, D. (2019). CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases, In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 1962–1979).

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* Brussels, Belgium.

Yu, T., Zhang, R., Yasunaga, M., Tan, Y. C., Lin, X. V., Li, S., Heyang Er, I. L., Pang, B., Chen, T., Ji, E., Dixit, S., Proctor, D., Shim, S., Jonathan Kraft, V. Z., Xiong, C., Socher, R., & Radev, D. (2019). SParC: Cross-Domain Semantic Parsing in Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* Florence, Italy:.

Zhang, S., Tan, Z., Yu, J., Zhao, Z., Kuang, K., Jiang, T., Zhou, J., Yang, H., & Wu, F. (2020). Comprehensive information integration modeling framework for video titling. CoRR, abs/2006.13608, URL: arXiv:2006.13608.

Zhang, R., Yu, T., Er, H., Shim, S., Xue, E., Lin, X. V., Shi, T., Xiong, C., Socher, R., & Radev, D. R. (2019). Editing-based SQL query generation for cross-domain context-dependent questions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019* (pp. 5337–5348). Association for Computational Linguistics.

Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating structured queries from natural language using reinforcement learning. CoRR, abs/1709.00103, URL: http://arxiv.org/abs/1709.00103.